

MODUL VIII

Integrasi UI dan Consume API

Tujuan

- ◆ Menyiapkan React app di direktori terpisah dari Laravel.
- ◆ Mengonfigurasi CORS di Laravel agar React (<http://localhost:5173>) dapat request API.
- ◆ Membuat komponen React untuk CRUD Items via axios.
- ◆ Menjalankan kedua server (React & Laravel) secara paralel.
- ◆ Membangun dan deploy tiap-tiap secara independen.

Materi

1. Arsitektur Front-End / Back-End Terpisah
 - Single-Page Application (SPA) di domain/port sendiri (misal <http://localhost:5173>)
 - Back-end Laravel API di domain/port lain (<http://localhost:8000>)
 - Komunikasi via HTTP/JSON, otentikasi token (Sanctum)
2. Cross-Origin Resource Sharing (CORS)
 - 2.1. Masalah SOP (Same-Origin Policy): browser memblokir request JS antar origin
 - 2.2. CORS header:
 - Access-Control-Allow-Origin
 - Access-Control-Allow-Methods
 - Access-Control-Allow-Headers
 - 2.3. Preflight request (OPTIONS)
 - 2.4. Konfigurasi di Laravel (config/cors.php & HandleCors middleware)
 - 2.5. Allowed_origins, credentials, max_age
3. ReactJS Dasar untuk SPA
 - Create-React-App / Vite setup
 - Function components & JSX
 - State (useState) & efek samping (useEffect)
 - Props untuk passing data
4. Routing di React dengan React Router
 - BrowserRouter, Routes, Route
 - Link / NavLink untuk navigasi tanpa reload
 - URL parameter (useParams)
5. HTTP Client: axios atau fetch
 - GET/POST/PUT/DELETE ke API

- Menyertakan header Authorization (token) jika perlu
 - Penanganan response & error (try/catch, .catch)
6. Konfigurasi Environment di React
 - .env dengan REACT_APP_API_URL
 - Memisahkan URL development vs production
 7. UX Minimum di React
 - Loading indicator / spinner saat fetch
 - Men-disable tombol saat submitting
 - Menampilkan validasi / error message di form
 - Feedback success/toast setelah aksi CRUD
 8. Build & Deployment Terpisah
 - 8.1. Front-End React
 - npm run build → folder build/ (static assets)
 - Deploy ke Netlify / Vercel / S3 + CloudFront
 - 8.2. Back-End Laravel
 - Composer install, migrate, config:cache
 - Serve via VPS/Nginx/Heroku
 - 8.3. CORS & Environment di production
 9. Error Handling & Debugging di Cross-Origin
 - Console network tab (CORS errors)
 - Axios interceptors untuk catch global
 - Logging di Laravel untuk request CORS gagal

Praktikum

1. Pastikan Anda switch dari branch main ke branch feature/cors:


```
cd C:\laragon\www\inventory
git checkout main
git pull origin main
git checkout -b feature/cors
```
2. Konfigurasi CORS di Laravel
 - 2.1. Publish config (jika belum ada):


```
php artisan vendor:publish \
  --provider="Fruitcake\Cors\CorsServiceProvider"
```
 - 2.2. Buka file config/cors.php, set:


```
'paths' => ['api/*'],
'allowed_methods' => ['*'],
'allowed_origins' => ['http://localhost:5173'],
'allowed_headers' => ['*'],
'supports_credentials' => false,
```

2.3. Pastikan middleware CORS di-load:

File `app/Http/Kernel.php`, di `$middleware` atau `$middlewareGroups['api']` sudah ada:

```
\Fruitcake\Cors\HandleCors::class,
```

2.4. Commit perubahan:

```
git add .
git commit -m "Configure CORS for React UI"
git push -u origin feature/cors
```

3. Pull Request

3.1. Buka Repository di GitHub

- Pergi ke `https://github.com/USERNAME/inventory`
- Kamu akan melihat notifikasi kuning: "**feature/cors had recent pushes**" → klik "**Compare & pull request**"

3.2. Isi Form Pull Request

- Base branch: main (branch tujuan merge)
- Compare branch: feature/auth-sanctum (branch kamu)
- Title: Isi judul PR
- Setelah form diisi, klik tombol "Create pull request"

3.3. Review Kode (Melihat Perubahan)

- Klik tab "Files changed"
- Lihat perubahan kode:
 - Warna hijau = baris kode yang ditambah
 - Warna merah = baris kode yang dihapus

3.4. Merge Pull Request

- Klik tombol "Merge pull request"
- Klik "Confirm merge"

4. Buat React app terpisah

Pindah ke folder parent di luar Laravel:

```
cd C:\laragon\www
npx create-react-app inventory-frontend
cd inventory-frontend
npm install axios react-router-dom bootstrap
```

Inisialisasi git (opsional):

```
git init
git remote add origin git@github.com:username/inventory-frontend.git
git add .
git commit -m "Initial React frontend"
git push -u origin main
```

5. Konfigurasi environment React

Buat file inventory-frontend/.env:

REACT_APP_API_URL=http://localhost:8000/api/v1

(otomatis load variabel ber-prefix REACT_APP_)

6. Bangun komponen CRUD

6.1. Struktur src/:

```
src/
├── components/
│   ├── ItemList.js
│   └── ItemForm.js
├── App.js
└── index.js
```

6.2. src/components/ItemList.js:

```
import React, { useEffect, useState } from 'react';
import axios from 'axios';
import { Link } from 'react-router-dom';
export default function ItemList() {
  const [items, setItems] = useState([]);
  const [loading, setLoading] = useState(true);
  const fetchItems = () => {
    axios.get(`${process.env.REACT_APP_API_URL}/items`)
      .then(res => setItems(res.data.data))
      .catch(console.error)
      .finally(() => setLoading(false));
  };
  const deleteItem = id => {
    if (!window.confirm('Delete this item?')) return;
    axios.delete(
      `${process.env.REACT_APP_API_URL}/items/${id}`
    ).then(fetchItems).catch(console.error);
  };
  useEffect(fetchItems, []);
  if (loading) return <p>Loading...</p>;
  if (items.length === 0) return <p>No items.</p>;
  return (
    <table className="table">
      <thead>
        <tr>
          <th>ID</th>
          <th>Name</th>
          <th>Qty</th>
          <th>Price</th>
          <th>Category</th>
          <th>Actions</th>
        </tr>
      </thead>
      <tbody>
        {items.map(i=>(
          <tr key={i.id}>
            <td>{i.id}</td>
```

```

        <td>{i.name}</td>
        <td>{i.quantity}</td>
        <td>{i.price}</td>
        <td>{i.category.name}</td>
        <td>
            <Link
                to={`/edit/${i.id}`}
                className="btn btn-sm btn-warning me-1"
            >
                Edit
            </Link>
            <button
                onClick={()=>deleteItem(i.id)}
                className="btn btn-sm btn-danger"
            >
                Del
            </button>
        </td>
    </tr>
</tbody>
</table>
    );
}

```

6.3. src/components/ItemForm.js:

```

import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { useNavigate, useParams } from 'react-router-dom';
export default function ItemForm() {
    const [form, setForm] = useState({
        name:'', quantity:'', price:'', category_id:''
    });
    const [errors, setErrors] = useState({});
    const [categories, setCategories] = useState([]);
    const navigate = useNavigate();
    const { id } = useParams();
    const isEdit = Boolean(id);
    useEffect(()=>{
        axios.get(`${process.env.REACT_APP_API_URL}/categories`)
            .then(res=>setCategories(res.data.data));
        if (isEdit) {
            axios.get(
                `${process.env.REACT_APP_API_URL}/items/${id}`
            ).then(res=>setForm(res.data.data));
        }
    },[id, isEdit]);
    const handleChange = e => {
        setForm({...form, [e.target.name]: e.target.value });
    };
    const handleSubmit = e => {
        e.preventDefault();
        setErrors({});
        const req = isEdit
            ? axios.put(

```

```

        `${process.env.REACT_APP_API_URL}/items/${id}`,
        form)
      : axios.post(
        `${process.env.REACT_APP_API_URL}/items`,
        form);
    req.then(() => navigate('/'))
      .catch(err => {
        if (err.response.status === 422)
          setErrors(err.response.data.data || {});
      });
  };
  return (
    <form onSubmit={handleSubmit}>
      <div className="mb-3">
        <label>Name</label>
        <input
          name="name"
          value={form.name}
          onChange={handleChange}
          className={`form-control ${
            errors.name ? 'is-invalid' : ''
          }`}
        />
        {errors.name &&
          <div className="invalid-feedback">
            {errors.name[0]}
          </div>
        }
      </div>
      {/* similarly quantity, price */}
      <div className="mb-3">
        <label>Category</label>
        <select
          name="category_id"
          value={form.category_id}
          onChange={handleChange}
          className={`form-control ${
            errors.category_id ? 'is-invalid' : ''
          }`}
        >
          <option value="">-- Select --</option>
          {categories.map(c => (
            <option
              key={c.id}
              value={c.id}
            >{c.name}</option>
          ))}
        </select>
        {errors.category_id &&
          <div className="invalid-feedback">
            {errors.category_id[0]}
          </div>
        }
      </div>
      <button type="submit" className="btn btn-primary">

```

```

        {isEdit ? 'Update' : 'Create'}
      </button>
    </form>
  );
}

```

6.4. src/App.js:

```

import React from 'react';
import {
  BrowserRouter,
  Routes,
  Route,
  Link } from 'react-router-dom';
import 'bootstrap/dist/css/bootstrap.min.css';
import ItemList from './components/ItemList';
import ItemForm from './components/ItemForm';
export default function App() {
  return (
    <BrowserRouter>
      <div className="container mt-4">
        <nav className="mb-3">
          <Link
            to="/"
            className="btn btn-outline-primary me-2"
          >Items</Link>
          <Link
            to="/create"
            className="btn btn-outline-success"
          >Add Item</Link>
        </nav>
        <Routes>
          <Route path="/" element={<ItemList />} />
          <Route path="/create" element={<ItemForm />} />
          <Route path="/edit/:id" element={<ItemForm />} />
        </Routes>
      </div>
    </BrowserRouter>
  );
}

```

6.5. Commit front-end:

```

git add .
git commit -m "Implement ItemList & ItemForm components
with React Router"
git push -u origin main

```

7. Jalankan kedua server

Backend:

```

cd C:\laragon\www\inventory
php artisan serve

```

Frontend:

```

cd C:\laragon\www\inventory-frontend
npm start

```

Buka browser ke <http://localhost:5173>, React UI berkomunikasi ke API di <http://localhost:8000/api/v1>.

8. Build & Deploy terpisah

8.1. Backend deploy (misal Heroku/VPS):

- push ke GitHub, setup Heroku remote, jalankan `git push heroku main`
- Heroku akan `composer install`, `php artisan migrate --force`

8.2. Frontend deploy (misal Netlify/Vercel):

- push React repo ke GitHub
- di Netlify: New site
 - link repo `inventory-frontend`
 - build command `npm run build`, publish `build/`
- Netlify akan host SPA di url custom.

9. Verifikasi production

- React app di URL Netlify (misal <https://inventory-ui.netlify.app>)
- Fetch API dari backend production
- Pastikan CRUD berfungsi.